

BookBot

CADTech AI Knowledge Assistant

College of Arts & Design · Rochester Institute of Technology

Project Pitch & Technical Overview

Prepared by	Dewansh Sharma — System Administrator Co-op, College of Arts & Design
Version	v1.0 — August 2026
Status	Functional prototype — ready for team deployment

1. What Is BookBot?

BookBot is an AI-powered chatbot built specifically for the CADTech IT team at RIT's College of Arts & Design. It allows any tech team member to ask questions in plain English and instantly receive answers sourced directly from your BookStack documentation — the same knowledge base your team has been building for years.

Instead of spending 5–10 minutes manually searching BookStack for a procedure or troubleshooting guide, a tech can type a question and get a complete, step-by-step answer in seconds.

The Problem It Solves

- Tech staff waste time searching BookStack manually for procedures they know exist
- Search results in BookStack require knowing exact keywords — natural language queries often fail
- New student workers have no way to discover institutional knowledge quickly
- Repeated questions go to senior staff who have to look up the same documentation

The Solution

- Ask BookBot in plain English: "machine won't display" or "how to set up a new workstation"
- BookBot searches your BookStack docs semantically — understanding meaning, not just keywords
- If the answer is in BookStack, it returns the exact steps from your documentation
- If not, it provides a general answer and offers a web search for additional resources

2. How It Works — Technical Overview

BookBot uses a technique called Retrieval-Augmented Generation (RAG) — the industry standard architecture for building AI chatbots that answer from private knowledge bases. Here is the pipeline in plain terms:

Step 1 — Ingestion (runs once, or when docs are updated)

- All BookStack pages are exported as plain text files
- Each page is split into small overlapping chunks of ~500 characters
- Each chunk is converted into a vector (a list of 768 numbers representing its meaning)
- All vectors are stored in ChromaDB — a local vector database on your machine

Step 2 — Query (every time a user asks a question)

- The user's question is converted into a vector using the same embedding model
- ChromaDB finds the 3 most semantically similar chunks from your BookStack docs
- Those chunks are passed to the local AI model (Mistral 7B) as context
- The model generates a human-readable answer using only that context

Step 3 — Fallback

- If no relevant documentation is found, BookBot says so clearly
- A "Search the web instead?" button appears for the tech to find external resources

Technology Stack

LLM	Mistral 7B via Ollama — runs locally, no internet required
Embeddings	nomic-embed-text via Ollama — converts text to vectors locally
Vector DB	ChromaDB — stores and searches document embeddings on disk
Orchestration	LangChain — manages chunking and retrieval pipeline

UI	Streamlit — browser-based chat interface, RIT branded
Web Fallback	Google Search — fallback when BookStack has no answer

3. Security & Privacy

Security was a primary design requirement. BookBot is architected so that your documentation never leaves RIT's network and no external AI services are used.

Concern	How BookBot addresses it
Does documentation leave RIT?	No. All documents are stored locally in ChromaDB on the host machine. Nothing is sent to any external server.
Is the AI model cloud-based?	No. Mistral 7B runs entirely via Ollama on the local machine. No internet connection is required for inference.
Who can access the chatbot?	Only devices on the RIT internal network can reach the Streamlit app. It is not exposed to the public internet.
Is there any API key or account needed?	No external API keys. No accounts. No subscriptions. Fully self-contained and free.
What if the internet goes down?	BookStack answers still work. Only the web search fallback requires internet — core RAG is fully offline.

Summary

BookBot is as secure as running a local application on your office computer. There are no cloud dependencies, no third-party AI APIs, no data transmission, and no accounts required. Your BookStack documentation stays on your hardware.

4. Deployment Options for RIT

Option A — Office Machine on Local Network (Recommended — Start Here)

Run BookBot on one dedicated office desktop. Everyone on the tech team accesses it through their browser via the RIT internal network. No server required, no IT request needed, deployable today.

Setup

- Install Ollama and pull Mistral 7B on the designated office machine
- Run: `streamlit run app.py --server.address 0.0.0.0 --server.port 8501`
- Share the internal IP address with the team: `http://192.168.X.X:8501`
- Machine must remain powered on during working hours

Pros

- Zero cost, zero infrastructure, working within hours
- Completely inside RIT network — not accessible from outside
- No IT department approval needed
- Easy to update docs — just re-run `ingest.py`

Cons

- Dependent on one machine staying on
- Speed depends on that machine's hardware (GPU recommended)
- Not accessible off-campus without VPN

Option B — RIT Internal Server with Domain (Long-Term)

Request a VM or internal server from RIT ITS. Deploy BookBot there with a proper internal URL such as `bookbot.rit.edu` or `cadtech.rit.edu/bookbot`. Accessible on campus and via RIT VPN off-campus.

Setup

- Submit a VM request to RIT ITS for an internal Linux server
- Install Ollama, Python, and dependencies on the server
- Configure Nginx as a reverse proxy to serve the Streamlit app
- Request an internal subdomain from RIT ITS (e.g. `bookbot.rit.edu`)
- Restrict access to RIT network / VPN only at the firewall level

Pros

- Professional internal URL — feels like a real RIT tool
- Always on — no dependency on an office desktop
- Accessible via RIT VPN for off-campus use
- Scalable — can serve the whole college

Cons

- Requires ITS approval and provisioning time
- Slightly more setup and configuration
- GPU server preferred for speed — may require budget justification

Recommendation

Start with Option A immediately to demonstrate value to the team. Once the team is using it regularly, use that adoption data to justify the ITS server request for Option B. Both options keep all data inside RIT's network.

5. AI Model Comparison

BookBot uses two AI models — one for understanding text (embedding model) and one for generating answers (language model). The embedding model is fixed. The language model can be swapped depending on your hardware.

Embedding Model — Fixed

nomic-embed-text	274 MB · Fast · Required · Converts text to vectors for semantic search · Cannot be changed without re-indexing all documents
------------------	---

Language Models — Interchangeable

Model	Size	Speed	Quality	Instruction Following	Recommended Use
llama3.2:3b	2 GB	Fastest	Low	Poor	Quick demos only
mistral:7b ★	4.1 GB	Medium	High	Very Good	Current — recommended
llama3.1:8b	4.9 GB	Slowest	Highest	Good	Best quality, slow Mac
phi3:mini	2.3 GB	Fast	Medium	Good	Low-RAM machines

★ Mistral 7B is the current recommended model. It provides the best balance of speed, answer quality, and instruction-following for IT troubleshooting use cases on standard office hardware.

Switching Models

To switch the language model, change one line in chatbot.py and restart the app. No re-indexing required. The embedding model and ChromaDB database are unaffected by language model changes.

6. Customization Options

BookBot is fully customizable for RIT's branding and operational requirements. The following elements can be changed without touching the core AI pipeline.

Visual Customization

- Primary color: Currently RIT Orange (#F76902) — can be changed to any hex color in config.toml
- Logo: Currently uses RIT logo — can be swapped to CADTech or department-specific logo
- App title: "BookBot - CADTech IT Assistant" — editable in app.py
- Subtitle: "CADTech IT Assistant — College of Arts & Design" — editable in app.py
- Footer text: "Powered by RIT CADTech · BookBot v1.0" — editable in app.py
- Dark/light mode: Currently dark only — can add light mode toggle

Behavioral Customization

- AI model: Swap Mistral 7B for any Ollama-compatible model in one line
- Chunk size: Adjust how documents are split (currently 500 characters, 50 overlap)
- Number of results: Currently retrieves top 3 chunks — can increase for more context
- Relevance threshold: Currently 370 distance — tune to make BookBot more or less strict
- System prompt: Adjust the AI's persona, tone, and instructions in chatbot.py

Data Customization

- Add more BookStack pages: Export as .txt and re-run ingest.py
- BookStack API: Swap local files for live API once admin token is available — one file change
- Multiple knowledge bases: Can add non-BookStack docs (PDFs, Word files) with minor modifications
- Auto-refresh: Can schedule ingest.py to run nightly to keep docs current

Access Customization

- Network restriction: Limit to specific IP ranges for added security
- Password protection: Add basic authentication via Streamlit or Nginx
- User logging: Add query logging to track which questions are asked most frequently

7. README — Setup & Installation

Prerequisites

- macOS or Linux machine (Apple Silicon supported)
- Python 3.11+ with pyenv recommended
- Ollama installed from ollama.com
- 8GB+ RAM recommended (16GB for best performance)

Installation

Step 1 — Clone the repository and create a virtual environment:

```
python3 -m venv venv && source venv/bin/activate
```

Step 2 — Install Python dependencies:

```
pip3 install -r requirements.txt
```

Step 3 — Pull AI models via Ollama:

```
ollama pull mistral:7b && ollama pull nomic-embed-text
```

Step 4 — Add BookStack exports:

- Export BookStack pages as plain text (.txt)
- Place all .txt files in the `bookstack_exports/` folder

Step 5 — Run the ingestion pipeline:

```
python3 ingest.py
```

Step 6 — Launch the app:

```
streamlit run app.py
```

Project Structure

```
BookBot/
├── bookstack_exports/  ← BookStack .txt files
├── data/               ← ChromaDB vector index
├── .streamlit/         ← Theme config
├── ingest.py           ← Load, chunk, embed, store
├── retriever.py        ← Semantic search
├── chatbot.py          ← LLM + web fallback
├── app.py              ← Streamlit UI
└── requirements.txt    ← Python dependencies
```

Daily Use

- Start Ollama: it runs automatically after installation
- Activate venv: `source venv/bin/activate`
- Launch: `streamlit run app.py`
- Add new docs: drop .txt files in `bookstack_exports/` and re-run `ingest.py`

8. Next Steps

BookBot is currently a functional prototype running on a single machine. The following steps will move it toward full production deployment for the CADTech team:

Immediate (this week)

- Export remaining BookStack shelves and add to bookstack_exports/
- Re-run ingest.py to index all documentation
- Deploy on a dedicated office machine for team testing

Short Term (this month)

- Obtain BookStack API token from system administrator
- Swap local file ingestion for live BookStack API integration
- Schedule automatic nightly re-ingestion to keep docs current

Long Term

- Submit RIT ITS request for internal VM with bookbot.rit.edu subdomain
- Add query logging to identify documentation gaps
- Expand to other RIT departments if adoption is successful